

Multithreaded Producer–Consumer System with Deadlock Detection

Sakarya University – Software Engineering Department

Operating Systems Course Project Report

Students:

Faruk Turnalı – B231202060

Ahme Hilmi Güler – B231202013

Hüseyin Yılmaz – B231202023

1. Introduction

Concurrent programming is one of the most important concepts in modern operating systems and software systems. Applications such as operating systems, database systems, web servers, task schedulers and data processing pipelines rely heavily on multithreading and synchronization mechanisms in order to execute multiple tasks simultaneously.

In concurrent systems, multiple threads may attempt to access shared resources at the same time. Without proper synchronization, this may lead to race conditions, inconsistent data, deadlocks and performance degradation. Therefore, synchronization and resource management are critical concepts in operating system design.

The aim of this project is to design and implement a configurable multithreaded producer–consumer system using bounded shared buffers. The project also includes synchronization mechanisms, deadlock detection, performance analysis and workload experiments.

The implemented system supports:

- Multiple producer and consumer threads
- Shared bounded buffers
- Mutex and condition variable synchronization
- Configurable thread structures using configuration files
- Deadlock detection using circular wait analysis
- Performance metric collection and evaluation
- Multi-buffer pipeline structures

The system was developed in the C programming language using POSIX Threads (pthreads).

2. Design Approach

2.1 General Architecture

The project was designed using a modular structure. Each component of the system was separated into independent modules in order to improve readability, maintainability and extensibility.

The main modules are:

Module	Responsibility
producer.c	Producer thread operations
consumer.c	Consumer thread operations
buffer.c	Shared bounded buffer operations
config.c	Configuration file parsing
utils.c	Logging, metrics and deadlock tracking
main.c	System initialization and thread management

The application workflow is:

1. Read configuration file
2. Initialize shared buffers
3. Create producer and consumer threads
4. Execute synchronized producer–consumer operations
5. Monitor deadlock situations
6. Collect performance metrics
7. Terminate system

2.2 Producer–Consumer Model

The system is based on the classical producer–consumer problem.

Producers

Producer threads generate integer values periodically and insert them into shared bounded buffers.

Each producer:

- Generates data at configured intervals
- Waits if the target buffer is full
- Signals consumers after inserting data
- Logs synchronization operations

Consumers

Consumer threads consume data from shared buffers.

Each consumer:

- Waits if the source buffer is empty
- Removes items from the buffer
- Simulates processing using sleep operations
- Optionally produces data into another buffer

Some consumers act as pipeline stages by consuming one type of component and producing another component.

Example:

B -> Consumer -> A

This structure enables circular dependency and pipeline experiments.

2.3 Shared Buffer Structure

The system uses bounded shared buffers implemented as circular queues.

Each buffer supports:

- Item insertion
- Item removal
- Full buffer detection
- Empty buffer detection

Each buffer contains:

- Integer array for items
- Front index
- Rear index
- Current item count
- Buffer capacity
- Mutex lock
- Condition variables

The buffer structure ensures controlled access to shared resources.

2.4 Configuration System

The application uses configuration files in order to create different runtime scenarios.

Configuration files define:

- Buffer sizes
- Runtime duration
- Producer connections
- Consumer connections
- Producer and consumer sleep intervals

The parser dynamically creates the thread structure according to the configuration file.

3. Synchronization Design

3.1 Synchronization Mechanisms

The system uses:

- pthread mutexes
- pthread condition variables

for synchronization.

Mutexes ensure mutual exclusion while accessing shared buffers. Condition variables are used for producer–consumer coordination.

The synchronization strategy prevents:

- Race conditions
- Simultaneous buffer corruption
- Unsafe concurrent access

3.2 Producer Synchronization

Before inserting data into a buffer, producers acquire the corresponding buffer mutex.

If the buffer is full, the producer waits using the **notFull** condition variable.

3.3 Consumer Synchronization

Consumers also use mutex and condition variables.

If the buffer is empty, consumers wait using the **notEmpty** condition variable.

3.4 Critical Section Protection

Critical sections include:

- Buffer insertion
- Buffer removal
- Shared metric updates
- Deadlock tracking structures

Mutex locks guarantee that only one thread accesses these sections at a time.

4. Deadlock Detection Method

4.1 Deadlock Concept

A deadlock occurs when multiple threads wait indefinitely for resources held by each other.

In this project, deadlock situations are created in Experiment 3.

The deadlock scenario was designed using circular wait conditions between two buffers.

4.2 Circular Wait Design

The artificial deadlock scenario works as follows:

1. Producer thread locks Buffer A
2. Consumer thread locks Buffer B
3. Producer waits for Buffer B
4. Consumer waits for Buffer A

This creates a circular wait condition.

4.3 Deadlock Monitoring

The system continuously monitors:

- Waiting threads
- Acquired resources
- Requested resources

The monitor thread periodically checks resource ownership and waiting dependencies.

The detection logic identifies circular wait situations between Buffer A and Buffer B.

When deadlock is detected:

- The system logs the deadlock event

- Runtime execution is terminated
- Waiting threads are released safely
- Performance metrics are printed

4.4 Deadlock Tracking Structures

The following information is tracked:

- Which thread owns Buffer A
- Which thread owns Buffer B
- Which resource each producer waits for
- Which resource each consumer waits for

These structures are protected using a dedicated tracking mutex.

5. Experiments

The system was tested under five different workloads.

Each experiment was executed for 60 seconds unless a deadlock occurred earlier.

5.1 Experiment 1 – Low Load

Configuration

- 2 Producers
- 2 Consumers
- Buffer A size: 6
- Producer interval: 10 ms
- Consumer interval: 10 ms

Results

```
===== PERFORMANCE METRICS =====  
Total produced: 2309  
Total consumed: 2303  
Producer wait count: 134  
Consumer wait count: 27  
Throughput: 38.38 item/sec  
Avg. Producer Wait Time: 12.84 ms  
Avg. Consumer Wait Time: 11.00 ms  
Total Producer Block Time: 10058 ms  
Total Consumer Block Time: 9974 ms  
Deadlock detected: NO  
Deadlock frequency: 0  
=====
```

Analysis

Under low workload conditions, the system operated efficiently with minimal waiting times and stable throughput.

Because thread contention was low, synchronization overhead remained small.

5.2 Experiment 2 – High Load

Configuration

- 10 Producers
- 10 Consumers
- Buffer A size: 20
- Producer interval: 10 ms
- Consumer interval: 10 ms

Results

```
===== PERFORMANCE METRICS =====  
Total produced: 2372  
Total consumed: 2369  
Producer wait count: 0  
Consumer wait count: 7  
Throughput: 39.48 item/sec  
Avg. Producer Wait Time: 0.00 ms  
Avg. Consumer Wait Time: 121.71 ms  
Total Producer Block Time: 493950 ms  
Total Consumer Block Time: 494981 ms  
Deadlock detected: NO  
Deadlock frequency: 0  
=====
```

Analysis

Increased thread count caused higher synchronization contention. Consumers spent significantly more time for waiting due to heavy concurrent access. Despite higher contention, the system maintained stable throughput.

5.3 Experiment 3 – Deadlock

Configuration

- 4 Producers
- 5 Consumers
- Buffer A size: 5
- Buffer B size: 5
- Artificial circular wait enabled

Results

```
===== PERFORMANCE METRICS =====
Total produced: 0
Total consumed: 0
Producer wait count: 0
Consumer wait count: 0
Throughput: 0.00 item/sec
Avg. Producer Wait Time: 0.00 ms
Avg. Consumer Wait Time: 0.00 ms
Total Producer Block Time: 0 ms
Total Consumer Block Time: 0 ms
Deadlock detected: YES
Deadlock frequency: 1
=====
```

Analysis

The system intentionally entered a circular wait state before any meaningful production or consumption operation could complete. The deadlock monitor successfully detected the circular wait condition between Buffer A and Buffer B. After detection, the system terminated safely without entering an infinite blocking state.

5.4 Experiment 4 – Bottleneck

Configuration

- 4 Producers
- 10 Consumers
- Buffer A size: 5
- Producer interval: 50 ms
- Consumer interval: 10 ms

Results

```
===== PERFORMANCE METRICS =====
Total produced: 2482
Total consumed: 2481
Producer wait count: 0
Consumer wait count: 2483
Throughput: 41.35 item/sec
Avg. Producer Wait Time: 0.00 ms
Avg. Consumer Wait Time: 174.27 ms
Total Producer Block Time: 40955 ms
Total Consumer Block Time: 58373 ms
Deadlock detected: NO
Deadlock frequency: 0
=====
```

Analysis

The slower producers created a bottleneck condition. Consumers frequently waited for new data because production speed was lower than consumption speed.

This experiment demonstrated starvation-like waiting behavior under producer limitations.

5.5 Experiment 5 – Circular Dependency

Configuration

- 2 Producers
- 10 Consumers
- Buffer A size: 5
- Buffer B size: 5
- Consumers both consume and produce components

Results

```
===== PERFORMANCE METRICS =====  
Total produced: 2404  
Total consumed: 2404  
Producer wait count: 0  
Consumer wait count: 2403  
Throughput: 40.07 item/sec  
Avg. Producer Wait Time: 0.00 ms  
Avg. Consumer Wait Time: 184.07 ms  
Total Producer Block Time: 5859 ms  
Total Consumer Block Time: 15985 ms  
Deadlock detected: NO  
Deadlock frequency: 0  
=====
```

Analysis

The circular dependency structure increased waiting times because components continuously moved between Buffer A and Buffer B. However, the system successfully avoided deadlock and continued execution. This experiment demonstrated stable concurrent execution under complex pipeline dependencies.

6. Performance Evaluation

The experiments demonstrate several important characteristics of concurrent systems.

6.1 Synchronization Overhead

As the number of threads increased, synchronization overhead also increased. High-load scenarios caused longer waiting and blocking times due to increased contention for shared resources.

6.2 Bottleneck Effects

Experiment 4 demonstrated that slower producers can create bottlenecks that force consumers to wait frequently. This reduced overall system efficiency.

6.3 Deadlock Behavior

Experiment 3 successfully demonstrated artificial circular wait deadlock creation. The implemented deadlock monitor correctly identified and terminated the deadlock scenario.

6.4 System Stability

Despite complex dependency structures, the system remained stable under all experiments except the intentionally induced deadlock case. The synchronization design prevented race conditions and protected shared resources correctly.

7. Conclusion

In this project, a configurable multithreaded producer–consumer system was successfully designed and implemented using POSIX threads.

The system supports:

- Concurrent producer and consumer execution
- Shared bounded buffers
- Synchronization using mutexes and condition variables
- Deadlock detection
- Configurable workloads
- Performance monitoring

The project demonstrated important operating system concepts such as:

- Thread synchronization
- Resource sharing
- Circular wait deadlocks
- Producer–consumer coordination
- Concurrent workload behavior

Experimental results showed that the system operates correctly under different workloads and successfully detects artificial deadlock scenarios.

Overall, the project provided practical experience with multithreaded programming, synchronization mechanisms and concurrent system analysis.